PulseLens — Project Document

> **Version:** 2.0
> **Purpose:** Full technical specification for building with Claude Code
> **Hackathon track:** Track 2 — Finance & Market Intelligence (+ Track 1 fit)
> **Tagline:** *See where markets are moving before reports catch up.*

Table of Contents

---

1. Mission and product definition

Core mission

> **Detect what is changing in a market — from web signals — before it appears in any official report.**

PulseLens is not a thermometer that only says "hot or cold." PulseLens is a **synthesizing analyst** that can tell you:

- Exactly *what* is changing
- *Why* it matters
- *Which companies* are affected and how
- *Which signals* to watch next

The timeline gap PulseLens fills

Buy-side analysts and strategy teams are living at Day 30–90. PulseLens pulls them back to Day 1–3.

What PulseLens is

Intelligence Dashboard + Analyst Chat for a specific vertical market.

- **Not** a trading terminal (not Bloomberg, not TradingView)
- **Not** a news aggregator (not RSS reader, not Google News)
- **Not** a generic chatbot (not ChatGPT with data)
- **Is** an alternative data engine with grounded intelligence — every claim traces back to a source

MVP scope

```
Market:       US AI Hardware / Semiconductor
Companies:    Nvidia, AMD, Intel, Broadcom, Supermicro, Dell, HPE, Micron
Time window:  Last 7 days
Data sources: News, job posts, pricing pages, product pages, SEC filings, IR pages
```

---

2. Target users and their problems

Primary users

User	Question they need answered	Current problem
Buy-side analyst (hedge fund, asset manager)	"Is this signal already priced in or is there still edge?"	Good alternative data exists but unstructured — takes hours to synthesize
Corporate strategy team	"Should we accelerate or delay expansion into this market?"	Reports are 30–90 days stale, no live signals
Finance / planning team	"Do we need to revise our forecast?"	Dependent on analyst reports that are already old
Market research analyst	"What does this week's sector brief say?"	Manual synthesis from dozens of sources

Secondary users (Track 1 fit)

User	Question they need answered
GTM / product marketing	"How is our competitor repositioning?"
Sales strategy	"Which sectors are showing buying momentum?"

What users do NOT want

- Another dashboard with pretty charts but hollow insights
- AI making things up without sourcing
- Reading hundreds of lines of text to find one insight
- Information they already knew last week

---

3. 4-layer intelligence framework

This is the most important mental model. PulseLens must deliver all 4 layers — not just Layer 1.

Layer → feature mapping

Layer	Feature	Location in UI
1	Pulse score card + status badge	Dashboard header, Overview tab
2	Company cards + signal breakdown + news feed + evidence table	Companies, Signals, News, Evidence tabs
3	Market narrative brief + cross-signal analysis + contradiction alerts	Overview tab, Analyst Chat
4	"What to watch" section + forward indicators + anomaly flags	Overview tab bottom, Analyst Chat

Layer 3 — concrete narrative example

Instead of:

> *"Pricing pressure detected. Confidence: 0.68"*

PulseLens says:

> *"Cloud GPU pricing fell 8–12% this week — but this is not necessarily a bad signal. This pattern historically precedes an enterprise demand surge 4–6 weeks after hyperscaler supply normalizes. Hiring momentum remains strong at Nvidia (+40%) and AMD (+34%), confirming the demand side has not weakened. Watch for enterprise AI server order announcements from Dell and HPE over the next two weeks."* [fact_d4] [fact_a1] [fact_a4]

Layer 4 — forward indicators example

```
WATCH LIST — Next week

1. Intel Q3 guidance revision  ← 3 signals converging, no official statement yet
   Trigger: 2 more negative hiring signals OR any IR page update

2. Supermicro audit resolution  ← Binary event, high impact either way
   Trigger: 10-K filing OR analyst comment on audit timeline

3. AMD MI400 roadmap  ← Hiring patterns suggest announcement imminent
   Trigger: Product page update OR job posts mentioning MI400
```

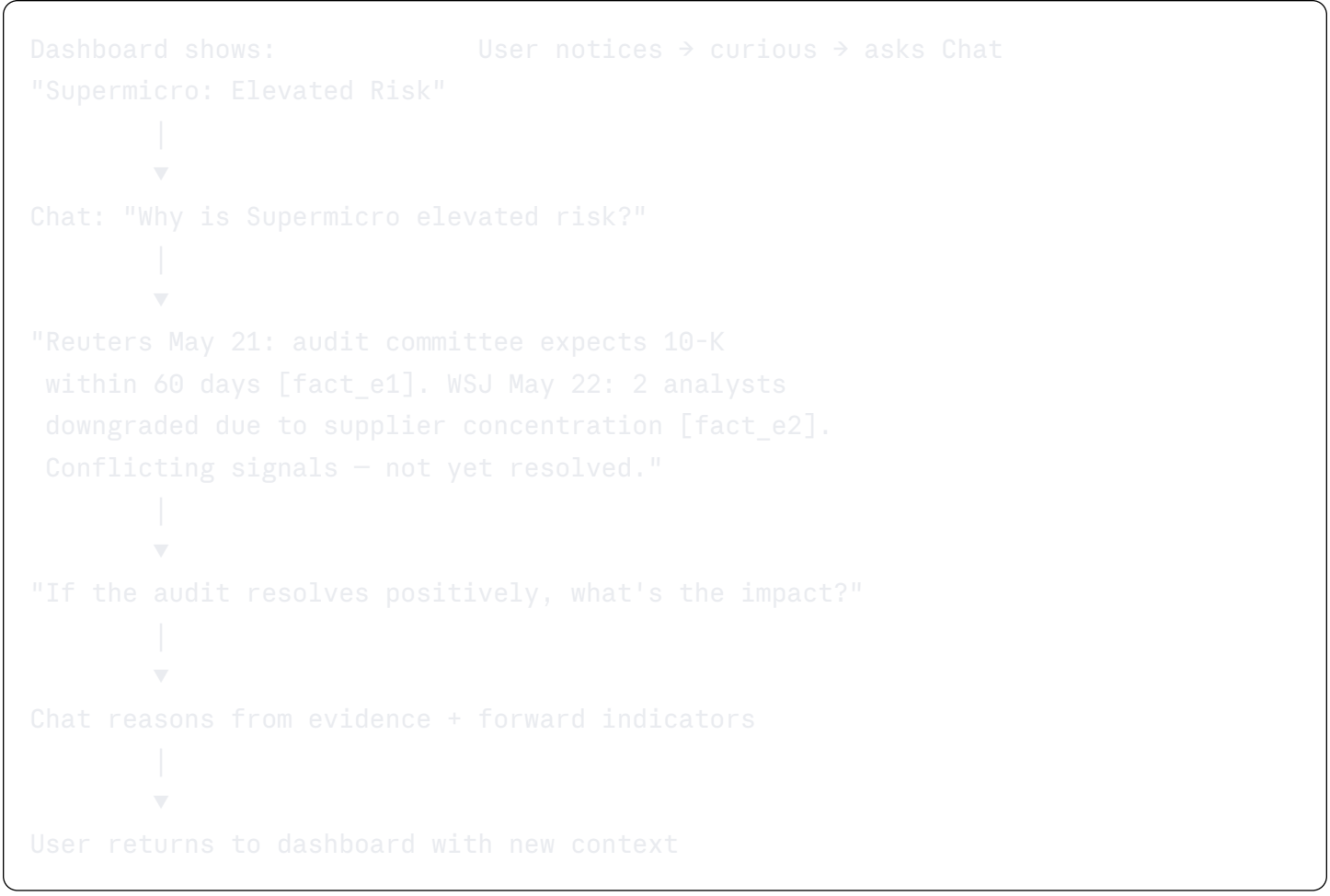
---

4. User interaction model

Design decision

> **PulseLens is an Intelligence Dashboard with an embedded Analyst Chat.** The dashboard is the default — users see it immediately on entry. Chat is for when users want to go deeper than what the dashboard shows.

These two parts are NOT separate features — they form a continuous loop:



3 primary user journeys

Journey 1 — Morning scan (5 minutes):

1. Open dashboard, check pulse score change vs last week
2. Scan top 5 signals — anything new?
3. Check contradiction alerts — any binary events pending?
4. Review "What to watch" — bookmark 1–2 items for follow-up

Journey 2 — Deep dive before a meeting (15 minutes):

1. Open Companies tab, read company narratives
2. Ask Chat: *"Compare Nvidia vs AMD hiring momentum over the last 2 weeks"*
3. Ask Chat: *"What signals support the thesis that AI server demand remains strong?"*
4. Export brief as Markdown for the presentation

Journey 3 — Specific research (open-ended):

1. Open Evidence tab, filter by company + signal type
2. Ask Chat a specific question about a company or event
3. Chat responds with citations — user clicks cite to see raw evidence
4. Ask follow-up questions to go deeper

Chat principles

1. **Only cite from triangulated facts** — no hallucination
2. **Every claim in response has [fact_id]** — user can verify
3. **Clearly separate "Found" vs "Infer"** — facts vs interpretations
4. **Refuse to answer if evidence is insufficient** — *"There is not enough data to conclude this"*
5. **Never predict stock prices** — PulseLens is a signal engine, not an oracle

---

5. System architecture

FRONTEND (FastAPI + Jinja2 / or Next.js)
Intelligence Dashboard
Sector Selection
Overview tab
Companies tab
Signals tab
News tab
Evidence tab
Analyst Chat Panel
Query input
Grounded responses
[fact_id] citations
"Found" vs "Infer" separation
Evidence link-through
API calls
RAG queries
BACKEND (FastAPI, Python)
/api/report
/api/run
/api/chat
RAG over fact_objects + verified_claims
PIPELINE
M1 Query Intelligence → M2 Web Collection (Bright Data)
M3 Structured Extraction (schema-constrained + FinBERT)
M4 Evidence Triangulation (corroboration + contradiction)
M5 Signal Scoring (weighted formula)
M6 Intelligence Output (all 4 layers + grounded brief)
SQLite Database
- fact_objects
- verified_claims
- market_reports
- chat_history
Alpha Vantage API
Stock price context layer
(signal lead time calc)

---

6. Data schemas

Design schemas before writing any code. These are the backbone of the entire system.

6.1 RawDocument

python

```python
class RawDocument(BaseModel):
    doc_id: str                    # uuid
    url: str
    domain: str
    title: str
    content: str                   # full text after HTML cleaning
    published_date: Optional[str]  # ISO 8601
    fetched_at: str                # ISO 8601
    source_tier: Literal[1, 2, 3, 4]
    collection_query: str          # the query used to fetch this document
    signal_type_hint: SignalType   # from query context
```

6.2 FactObject

python

```python
class FactObject(BaseModel):
    fact_id: str                    # uuid, format "fact_xxxx"
    doc_id: str                     # ref → RawDocument
    entity: str                     # "Nvidia", "AMD", "market"
    signal_type: SignalType
    claim: str                      # 1 factual sentence, max 150 chars
    evidence_quote: str             # exact quote from source, max 200 chars
    source_url: str
    source_tier: Literal[1, 2, 3, 4]
    published_date: Optional[str]
    sentiment: Literal["positive", "negative", "neutral"]
    sentiment_score: float          # FinBERT output, -1.0 to 1.0
    confidence: float               # LLM extraction confidence, 0.0-1.0

# MANDATORY validation rule:
# evidence_quote MUST appear verbatim in RawDocument.content
# If not → discard fact (LLM is hallucinating the quote)
```

6.3 VerifiedClaim

python

```python
class VerifiedClaim(BaseModel):
    claim_id: str
    entity: str
    signal_type: SignalType
    summary: str                    # 1 synthesized sentence from multiple facts
    supporting_facts: List[str]     # fact_id[]
    corroboration_count: int        # number of independent sources (distinct domains
    source_tiers_present: List[int]
    weighted_sentiment: float       # avg sentiment weighted by tier + recency
    recency_score: float            # 0.0-1.0
    final_confidence: float         # 0.0-1.0
    is_contradicted: bool
    contradiction_note: Optional[str]
```

6.4 CompanyNarrative (Layer 2)

```python
class CompanyNarrative(BaseModel):
    company: str
    ticker: str
    momentum: MomentumLabel
    momentum_score: int            # -100 to 100
    narrative: str                 # 3-5 analyst-grade sentences about this company
    key_events: List[str]          # specific events ("Updated IR page May 20")
    key_drivers: List[str]         # signal types driving momentum
    competitive_position: str      # "gaining" | "holding" | "losing" vs peers
    supporting_claim_ids: List[str]
    evidence_count: int
    # Stock price context (from Alpha Vantage)
    price_current: Optional[float]
    price_change_7d_pct: Optional[float]
    signal_lead_days: Optional[int] # how many days before price moved
```

6.5 MarketNarrative (Layer 3 + 4)

```python
class MarketNarrative(BaseModel):
    # Layer 3 — cross-signal causality
    narrative_headline: str        # 1 analyst-grade sentence
    narrative_body: str            # 3-5 sentences: context, causality, what's unusu
    anomalies: List[AnomalyFlag]

    # Layer 4 — forward indicators
    watch_list: List[WatchItem]    # 3-5 things to monitor next week

class AnomalyFlag(BaseModel):
    description: str               # "Hiring up + pricing down simultaneously = unus
    signal_types_involved: List[SignalType]
    implication: str
    fact_ids: List[str]

class WatchItem(BaseModel):
    title: str                     # "Intel Q3 guidance revision"
    rationale: str                 # why this matters
    trigger: str                   # "Triggered if: X happens"
    signals_pointing_there: List[str]  # fact_ids or claim_ids
    urgency: Literal["this_week", "next_2_weeks", "this_month"]
```

6.6 MarketPulseReport (final output)

python

```python
class MarketPulseReport(BaseModel):
    report_id: str
    market: str
    time_window: str
    generated_at: str

    # Layer 1
    pulse_score: float              # 0-100
    pulse_status: PulseStatus
    pulse_confidence: float
    trend_vs_previous: Optional[float]  # delta points vs last period

    # Layer 2
    top_signals: List[SignalSummary]
    company_narratives: List[CompanyNarrative]
    news_items: List[NewsItem]

    # Layer 3
    market_narrative: MarketNarrative
    contradictions: List[ContradictionFlag]
    grounded_brief: GroundedBrief  # what_we_found + what_we_infer + implication

    # Layer 4 (inside market_narrative.watch_list)

    # Meta
    evidence_count: int
    source_count: int
    signal_breakdown: Dict[str, float]  # signal_type → score

class GroundedBrief(BaseModel):
    what_we_found: List[CitedStatement]   # direct facts only
    what_we_infer: List[CitedStatement]   # interpretations, clearly labeled
    strategic_implication: str

class CitedStatement(BaseModel):
    text: str
    fact_ids: List[str]           # citations — at least 1 required
```

6.7 Enums

```python
from enum import Enum

class SignalType(str, Enum):
    HIRING_MOMENTUM    = "hiring_momentum"
    PRODUCT_LAUNCH     = "product_launch"
    PRICING_PRESSURE   = "pricing_pressure"
    STRATEGIC_MESSAGING = "strategic_messaging"
    INVESTOR_SIGNAL    = "investor_signal"
    NEWS_SENTIMENT     = "news_sentiment"
    SUPPLIER_RISK      = "supplier_risk"

class PulseStatus(str, Enum):
    HEATING_UP   = "heating_up"
    STABLE       = "stable"
    COOLING_DOWN = "cooling_down"
    VOLATILE     = "volatile"
    RISK_RISING  = "risk_rising"

class MomentumLabel(str, Enum):
    STRONG_POSITIVE = "strong_positive"
    POSITIVE        = "positive"
    NEUTRAL         = "neutral"
    MIXED           = "mixed"
    NEGATIVE        = "negative"
    ELEVATED_RISK   = "elevated_risk"
```

---

7. Backend — 6 pipeline modules

Module 1 — Query Intelligence

Goal: Turn 1 business question → 15–25 targeted sub-queries.

Method: Multi-HyDE (arXiv:2509.16369) — generate N non-equivalent queries to cover the full signal space.

Query decomposition across 3 dimensions:

- Company dimension: 1 query per company + 1 market-level query
- Signal type dimension: 1–3 queries per signal type (7 types)
- Source type dimension: news SERP, job search, IR pages, pricing pages

Query templates:

```python
QUERY_TEMPLATES = {
    "hiring_momentum": [
        "{company} AI infrastructure job openings {year}",
        "{company} hiring data center engineering roles",
        "{company} workforce expansion {quarter}",
    ],
    "product_launch": [
        "{company} new AI hardware product announcement {year}",
        "{company} product launch press release",
    ],
    "pricing_pressure": [
        "AI server pricing discount {year}",
        "{company} GPU price reduction competitor",
        "cloud GPU on-demand pricing {company}",
    ],
    "strategic_messaging": [
        "{company} AI strategy investor day {year}",
        "{company} CEO earnings call AI infrastructure",
    ],
    "investor_signal": [
        "{company} SEC 8-K filing {quarter} {year}",
        "{company} earnings guidance revision",
    ],
    "news_sentiment": [
        "{company} AI hardware news last 7 days",
        "{company} semiconductor market position {year}",
    ],
    "supplier_risk": [
        "{company} supply chain disruption {year}",
        "{company} supplier concentration risk",
    ],
}
```

Output: `List[SearchQuery]` with fields: query_id, query_text, target_entity, signal_type, source_type, priority, expected_source_tier.

Module 2 — Web Data Collection

Goal: Fetch raw content from the web. Assign source_tier at collection time — not later.

Source tiering:

Tier	Type	Weight	Examples
1	Official financial disclosures	1.0	SEC EDGAR, IR pages, earnings transcripts
2	Tier-1 financial/tech media	0.8	Reuters, Bloomberg, WSJ, FT
3	Specialist tech media	0.5	TechCrunch, Wired, Ars Technica, SemiAnalysis
4	Operational signals	0.4	LinkedIn jobs, pricing pages, company blogs

Bright Data tool mapping:

```python
TOOL_MAPPING = {
    "serp_news":      "SERP API",           # news, search results
    "job_pages":      "Web Scraper API",    # LinkedIn, Glassdoor, Indeed
    "ir_pages":       "Web Scraper API",    # SEC EDGAR, IR pages
    "pricing_pages":  "Web Scraper API",    # pricing, distributor listings
    "dynamic_pages":  "Scraping Browser",   # JS-rendered pages
    "protected":      "Web Unlocker",       # anti-bot protected sites
}
```

Domain-to-tier mapping (hardcoded for MVP):

```python
TIER_1_DOMAINS = {
    "sec.gov", "ir.nvidia.com", "ir.amd.com", "investor.intel.com",
    "investors.broadcom.com", "ir.supermicro.com", "ir.dell.com"
}
TIER_2_DOMAINS = {
    "reuters.com", "bloomberg.com", "wsj.com", "ft.com",
    "apnews.com", "businesswire.com", "prnewswire.com"
}
TIER_3_DOMAINS = {
    "techcrunch.com", "theverge.com", "wired.com",
    "semianalysis.com", "tomshardware.com", "anandtech.com"
}
# Tier 4 = default for all other domains
```

Implementation notes:

- Batch 5 queries per API call
- Timeout 15s, retry 3 times with exponential backoff
- Cache by (url, date) — never fetch the same URL twice in one run
- Save raw HTML to disk before extraction (for debugging)

Module 3 — Structured Fact Extraction

Goal: Raw documents → `FactObject[]` following a fixed schema.

Method: RASG — Retrieval Augmented Structured Generation (arXiv:2405.20245). Frame extraction as a tool-use task, not free-form summarization.

Extraction prompt:

```
System: You are a financial market intelligence extraction system.
Extract ONLY facts EXPLICITLY STATED in the provided text.
Do NOT infer, interpret, or add information not present in the text.
Return ONLY a valid JSON array. If no relevant facts found, return [].

Schema for each fact:
{
  "entity": "Company name or 'market'",
  "signal_type": "one of the 7 signal types",
  "claim": "1 factual sentence, max 150 chars, no interpretation",
  "evidence_quote": "exact quote from the text, max 200 chars",
  "published_date": "ISO 8601 or null",
  "confidence": 0.0-1.0
}

Context: query="{query}", expected_signal="{signal_type}"

Text:
{document_content}
```

Mandatory validation:

```python
def validate_fact(fact: FactObject, source: RawDocument) -> bool:
    # CRITICAL: evidence_quote must appear verbatim in source
    if fact.evidence_quote not in source.content:
        return False   # LLM hallucinated the quote — discard
    if len(fact.claim) > 150:
        return False
    if fact.confidence < 0.6:
        return False
    if fact.entity not in KNOWN_ENTITIES:
        return False
    return True
```

FinBERT sentiment scoring:

```python
# Model: ProsusAI/finbert (HuggingFace)
# Run on each fact.claim after extraction
# Output: sentiment label + score (-1.0 to 1.0)
# Use FinBERT instead of general LLM: trained on financial text,
# faster, cheaper for simple sentiment classification
```

Module 4 — Evidence Triangulation

Goal: Determine which claims are trustworthy enough to surface. Detect contradictions.

Method: ClaimCheck corroboration pattern (ACL 2025) + recency decay weighting.

Core rules:

- A claim is only raised if `corroboration_count >= 2` (≥2 independent domains)
- A Tier 1 source alone overrides the corroboration requirement
- Contradiction = same group has both positive and negative sentiment → do NOT blend, flag explicitly

Recency decay formula:

```python
def recency_weight(published_date: str, window_days: int = 7) -> float:
    age_days = days_since(published_date)
    if age_days > window_days:
        return 0.0
    return 1.0 / (age_days + 1)   # Day 0 = 1.0, Day 6 = 0.14

def weighted_sentiment(facts: List[FactObject]) -> float:
    # weight = source_tier_weight × recency_weight
    # return weighted average of sentiment_scores
    tier_weights = {1: 1.0, 2: 0.8, 3: 0.5, 4: 0.4}
    total_weight = sum(tier_weights[f.source_tier] * recency_weight(f.published_date
                       for f in facts)
    weighted_sum = sum(f.sentiment_score * tier_weights[f.source_tier] *
                       recency_weight(f.published_date) for f in facts)
    return weighted_sum / total_weight if total_weight > 0 else 0.0
```

Contradiction handling — important:

When a contradiction is detected, do NOT blend it into a neutral statement. Instead:

```python
def build_contradiction_note(facts: List[FactObject]) -> str:
    positive = [f for f in facts if f.sentiment == "positive"]
    negative = [f for f in facts if f.sentiment == "negative"]
    return (
        f"Conflicting signals: {len(positive)} source(s) report positive "
        f"({', '.join(extract_domain(f.source_url) for f in positive[:2])}), "
        f"while {len(negative)} source(s) report negative "
        f"({', '.join(extract_domain(f.source_url) for f in negative[:2])}). "
        f"Recommend manual review."
    )
```

Module 5 — Signal Scoring Engine

Goal: Calculate explainable scores from verified claims. Every score must be a function of actual evidence — not LLM opinion.

Signal weights:

python

```python
SIGNAL_WEIGHTS = {
    "investor_signal":      0.25,   # highest — direct financial impact
    "news_sentiment":       0.20,
    "pricing_pressure":     0.18,   # direct margin impact
    "strategic_messaging":  0.15,
    "hiring_momentum":      0.12,
    "product_launch":       0.07,
    "supplier_risk":        0.03,   # low weight but triggers status override
}
```

Pulse score formula:

python

```python
# signal_score = weighted avg sentiment of verified claims for that signal type
# Contradiction penalty: is_contradicted → weight × 0.5
# pulse_raw = weighted_avg(signal_scores, weights=SIGNAL_WEIGHTS)
# pulse_score = normalize(pulse_raw) from [-1,1] → [0,100]
# confidence = mean(final_confidence for all verified claims)
```

Pulse status classification:

python

```python
def classify_status(score: float,
                    has_supplier_risk: bool,
                    contradiction_rate: float) -> PulseStatus:
    if contradiction_rate > 0.4:
        return PulseStatus.VOLATILE
    if has_supplier_risk and score < 55:
        return PulseStatus.RISK_RISING
    if score >= 70:
        return PulseStatus.HEATING_UP
    if score >= 45:
        return PulseStatus.STABLE
    return PulseStatus.COOLING_DOWN
```

Company momentum ranking:

- momentum_score = mean(weighted_sentiment) of all claims about that company × 100
- Override to "elevated_risk" if any supplier_risk signal has sentiment < -0.3
- Sort: strong_positive → positive → neutral → mixed → negative → elevated_risk

Module 6 — Intelligence Output (all 4 layers)

Goal: Synthesize all evidence into a 4-layer intelligence report.

Layers 1–2: Pure logic from Module 5 output — no LLM needed.

Layer 3 — Market Narrative (LLM synthesis):

```
System: You are a senior market analyst writing for buy-side investment teams.
Synthesize the verified claims below into a market narrative.

Rules:
1. narrative_headline = 1 sentence, analyst-grade, no filler language
2. narrative_body = 3–5 sentences explaining WHY signals matter,
   cross-signal causality, what is unusual vs expected behavior
3. anomalies = patterns that don't fit normal expectations — flag these
4. Every sentence MUST reference at least one claim_id in brackets
5. Separate what the evidence shows from what it may mean
6. Never predict stock prices
7. If signals conflict, state this explicitly — do not smooth over it

Verified claims:
{verified_claims_json}

Signal scores:
{signal_scores_json}

Company rankings:
{company_rankings_json}

Return ONLY valid JSON matching the MarketNarrative schema.
```

Layer 4 — Watch List (LLM synthesis):

```
Based on the evidence above, identify 3–5 forward indicators.
For each item: what to watch, why it matters, what would trigger concern,
and which existing signals are pointing toward it.
Focus only on things that are currently DEVELOPING but not yet resolved.

Return as JSON array of WatchItem objects.
```

Grounded brief generation rules:

- `what_we_found` = direct facts from sources only, every sentence requires fact_id
- `what_we_infer` = interpretations, must begin with "This may...", "This could...", "Based on..."
- No sentence in `what_we_found` may lack a citation
- No stock price predictions anywhere in the output

---

8. Frontend — Web structure

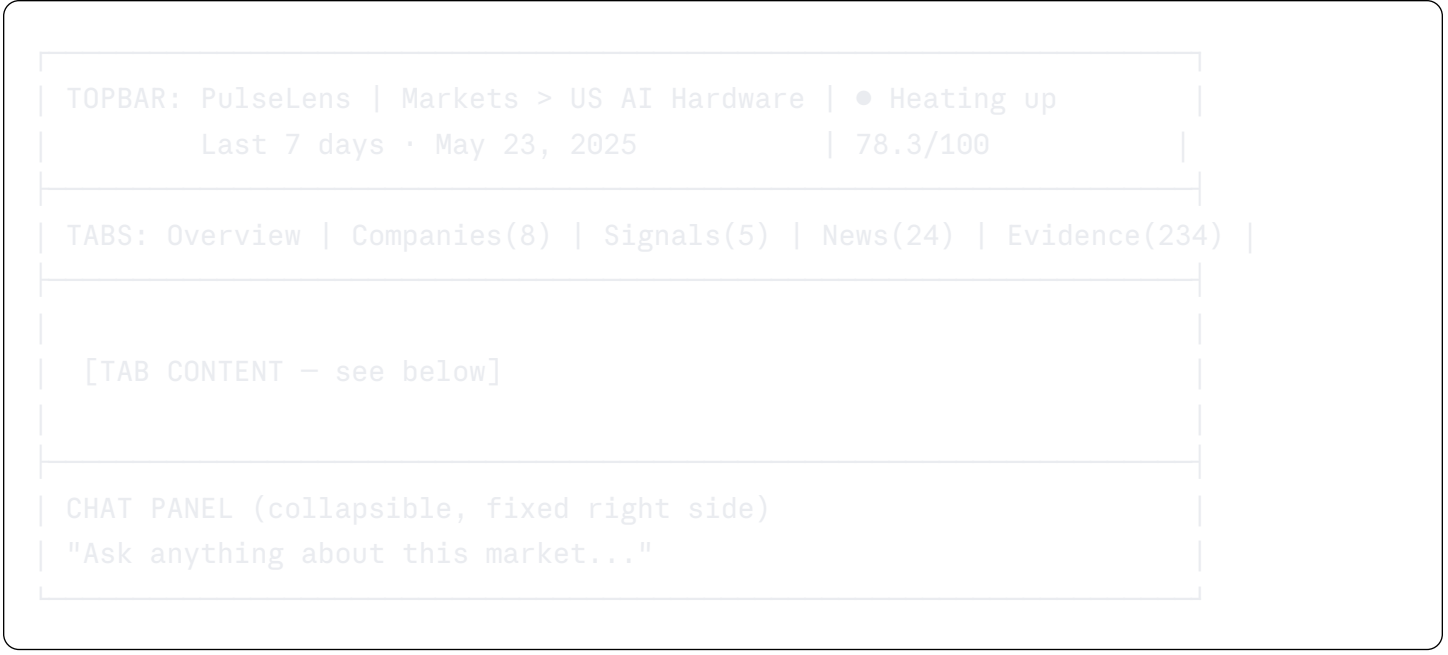
Screen 1: Sector Selection

The first thing users see — not the dashboard.

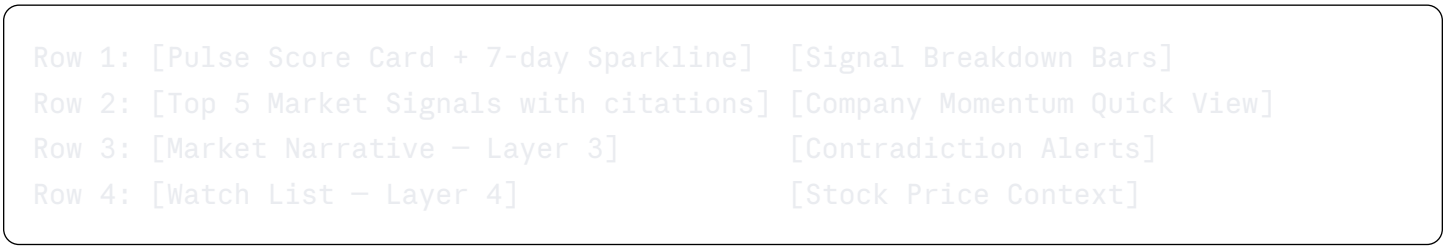
Active sectors (MVP): Only "US AI Hardware" is clickable.
Coming soon sectors: Grayed out, not clickable, "Coming soon" badge.

Screen 2: Intelligence Dashboard

Layout after sector selection:



Tab 1: Overview



Pulse Score Card:

- Large number (78.3) + status badge (Heating up)
- Trend vs previous period (+6.1)
- 7-day sparkline (inline SVG)
- Signal breakdown mini bars (one per signal type)

Market Narrative section (Layer 3):

- Headline — 1 analyst-grade sentence
- Body — 3–5 sentences with inline [fact_id] citations
- Anomaly flags if any detected

Watch List section (Layer 4):

- 3–5 items, each with: title, rationale, trigger condition, urgency badge (this week / next 2 weeks / this month)

Contradiction Alerts:

- One alert box per contradiction
- Shows both sides of the conflict with citations
- "Recommend manual review" label

Stock Price Context:

- Mini table: Company | Current Price | 7d Change% | Signal Lead Time
- "Signal detected X days before +Y% move" where calculable
- Disclaimer: *"Context only — not investment advice"*

Tab 2: Companies

2-column grid of 8 company cards.

Each company card:

```
[Company Initial Circle]  [Name]  [Ticker]       [Momentum Badge]
Momentum: [Score bar -100 to +100]  [Score number]
Drivers:  [tag] [tag] [tag]
──────────────────────────────────────────────────
[Company Narrative — 2–3 analyst-grade sentences]
[Key Event this week — 1 specific sentence]
[Stock: $892.40  ▲ +8.2% (7d)]
[47 facts · View evidence →]
```

Competitive landscape section (bottom of tab):

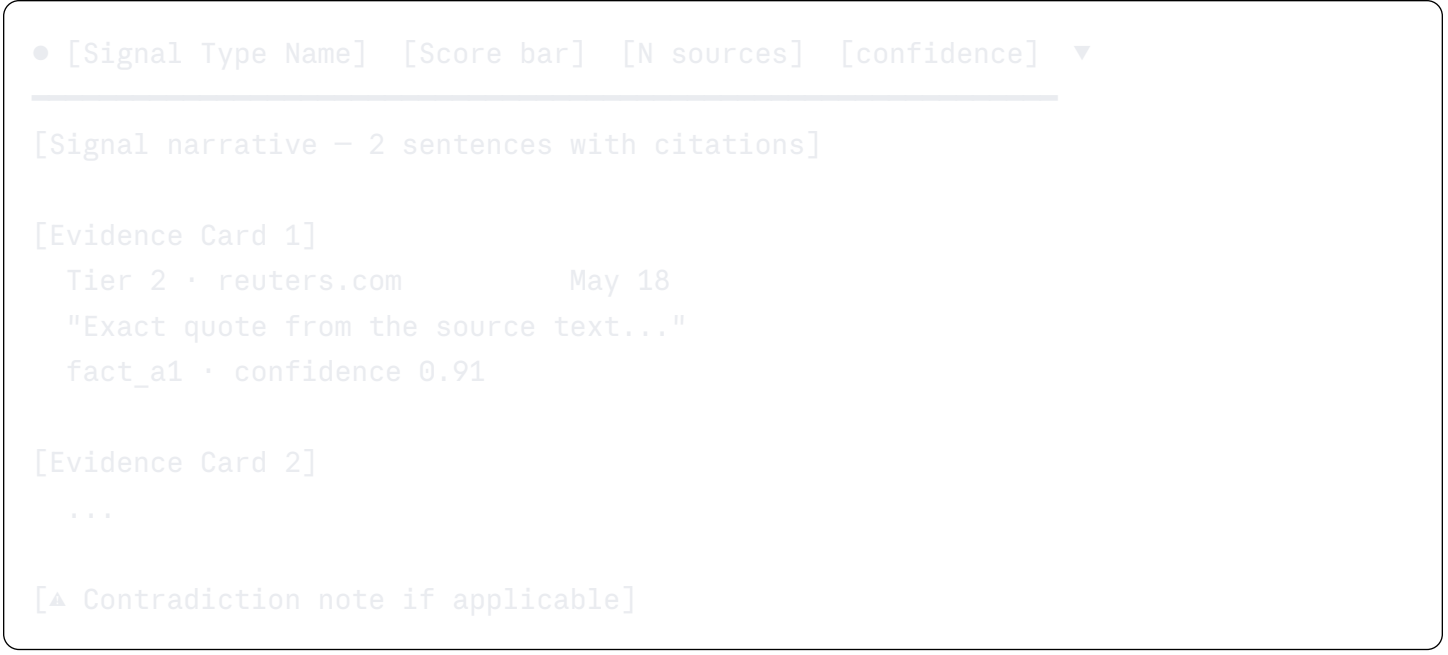
- A compact section comparing relative positions: who is gaining / holding / losing ground

---

Tab 3: Signals

5 collapsible sections, one per signal type.

Each signal section:



Tab 4: News

Chronological feed of all news items.

- Filter pills: All | Nvidia | AMD | Intel | Broadcom | Supermicro | Dell | HPE | Micron | + by signal type
- Each news item: Tier badge | Source domain | Date | Sentiment badge | Headline | 2-sentence summary | [fact_id]

Tab 5: Evidence

Full filterable table of all fact objects.

- Filter dropdowns: Company | Signal Type | Source Tier | Sentiment | Min Confidence
- Columns: fact_id | Entity | Signal Type | Claim | Source (with tier badge) | Date | Sentiment | Confidence
- Export to CSV button
- "Showing X of 234 facts · Load more"
- Clicking any fact_id highlights it and shows the full evidence_quote + source URL

---

9. Analyst Chat layer

Architecture

Chat does NOT run the pipeline again. It queries the evidence already in the database.

```python
# POST /api/chat
async def chat(query: str, report_id: str, history: List[dict]) -> ChatResponse:
    # 1. Load verified_claims and fact_objects from DB for this report
    # 2. RAG: embed query, find top-k relevant facts by similarity
    # 3. Build prompt with relevant evidence as context
    # 4. LLM generates response with citation constraints
    # 5. Validate: all fact_ids in response must exist in DB
    # 6. Return: response text + cited_facts (for frontend link-through)
```

Chat system prompt

```
You are PulseLens Analyst, a market intelligence assistant.
Answer questions about the US AI Hardware market using ONLY the verified
evidence from this week's intelligence report provided below.

STRICT RULES:
1. Every factual claim must cite [fact_id] from the provided evidence list.
2. If asked about something not in the evidence, respond:
   "I don't have sufficient evidence in this week's report to answer that."
3. Clearly separate facts from inferences:
   - Facts: cite [fact_id] directly after the statement
   - Inferences: begin with "This may suggest...", "Based on these signals..."
4. NEVER predict stock prices or give buy/sell recommendations.
5. NEVER fabricate fact_ids — only use IDs from the list below.
6. When signals conflict, present both sides explicitly and flag the
contradiction.

Available evidence this week:
{relevant_facts_json}

Market context:
Pulse score: {pulse_score} | Status: {pulse_status}
Time window: {time_window}
Companies tracked: {companies}
```

Chat example interactions

User: "Why is Supermicro elevated risk?"

Chat: "There are two conflicting signals about Supermicro this week. Reuters on May 21 reported that SMCI's audit committee expects to file the delayed 10-K within 60 days [fact_e1, confidence 0.71]. However, WSJ on May 22 reported that two analysts downgraded SMCI citing ongoing audit uncertainty and supplier concentration risk [fact_e2, confidence 0.85].

This may suggest this is a binary event — a positive resolution could unlock pent-up demand, while a negative one could spread supply chain risk more broadly. Manual review is recommended before drawing conclusions."

User: "What is the strongest signal this week?"

Chat: "The strongest signal is investor_signal at 88/100, driven by the Nvidia Blackwell GB200 ramp confirmation from three Tier 1–2 sources [fact_c1, fact_c2, fact_c3]. Corroboration count is 6 independent domains with confidence 0.88 — the highest in this report.

Hiring momentum (74/100) is also strong with corroboration from LinkedIn [fact_a1], Reuters [fact_a3], and TechCrunch [fact_a4]. *This may suggest* demand expansion is accelerating at the AI infrastructure layer."

Chat context management

```python
# Store chat history in DB per session
# Send last 5 exchanges as conversation context
# Max 2000 tokens for evidence context per request
# When overflow: keep most-cited facts + most recent exchanges
# Each response includes cited_fact_ids for frontend link-through to Evidence tab
```

---

10. Stock price integration

The correct role of stock price in PulseLens

Stock price is NOT the main feature. It is a **signal validation layer** — used to demonstrate:

> *"PulseLens detected this signal X days before the market reacted"*

This is proof of value for buy-side audiences. PulseLens is a signal engine, not a trading tool.

What to display

```python
class StockContext(BaseModel):
    company: str
    ticker: str
    price_current: float
    price_7d_change_pct: float
    price_7d_high: float
    price_7d_low: float
    # Key insight:
    signal_detected_date: Optional[str]  # date PulseLens detected the signal
    price_move_date: Optional[str]       # date price moved significantly (>3%)
    signal_lead_days: Optional[int]      # = price_move_date - signal_detected_date
    lead_time_note: Optional[str]        # "Signal detected 3 days before +8.2% move
```

What NOT to display

- Candlestick charts, technical indicators (RSI, MACD, Bollinger Bands)
- Buy / sell / hold recommendations
- Price targets or price predictions
- Real-time streaming prices

API: Alpha Vantage

```python
# Free tier: 25 requests/day — sufficient for 8 companies updated daily
# Endpoint: GLOBAL_QUOTE for current price, TIME_SERIES_DAILY for history
# Cache results for 4 hours to conserve quota
API_BASE = "https://www.alphavantage.co/query"
```

UI placement

- In each Company Card: one small line — `NVDA  $892.40  ▲ +8.2% (7d)`
- In Overview tab: "Stock Price Context" section at the bottom
  - Compact table: Company | Price | 7d% | Signal Lead Time
  - Framing disclaimer: *"Provided as context only — not investment advice"*

---

11. Tech stack and file structure

Stack

```
Backend:    Python 3.11, FastAPI, uvicorn
Database:   SQLite  (zero setup, sufficient for demo)
LLM:        Anthropic claude-sonnet-4-20250514
Sentiment:  transformers + ProsusAI/finbert
Web data:   Bright Data SDK
Stock:      Alpha Vantage API (free tier)
Cache:      diskcache
Frontend:   Jinja2 templates + vanilla JS + CSS
Deploy:     Railway (1 service, 1 command)
```

File structure

```
pulselens/
├── main.py                          # FastAPI app entry point, route registration
├── requirements.txt
├── .env.example
├── Dockerfile
├── railway.toml
├── PULSELENS_PROJECT.md             # this file
│
├── app/
│   ├── config/
│   │   ├── companies.py             # 8 companies: name, ticker, domain, ir_url,
careers_url
│   │   ├── signal_types.py          # SignalType enum + SIGNAL_WEIGHTS dict
│   │   └── source_tiers.py          # domain→tier mapping + assign_tier(url)
function
│   │
│   ├── schemas/
│   │   └── models.py                # all Pydantic v2 models from section 6
│   │
│   ├── pipeline/
│   │   ├── m1_query_intelligence.py
│   │   ├── m2_web_collection.py
│   │   ├── m3_fact_extraction.py
│   │   ├── m4_triangulation.py
│   │   ├── m5_scoring.py
│   │   └── m6_intelligence_output.py
│   │
│   ├── api/
│   │   ├── report.py                # GET /api/report/{id}, POST /api/run
│   │   ├── chat.py                  # POST /api/chat
│   │   └── stock.py                 # GET /api/stock/{ticker}
│   │
│   └── utils/
│       ├── brightdata_client.py     # Bright Data SDK wrapper
│       ├── llm_client.py            # Anthropic wrapper with retry + structured
logging
│       ├── finbert_client.py        # FinBERT sentiment pipeline
│       ├── alphavantage_client.py   # Alpha Vantage wrapper with caching
│       └── helpers.py               # uuid generation, date utils, text cleaning
│
├── templates/
│   ├── base.html                    # shared layout, topbar, navigation
│   ├── sector_select.html           # Screen 1: sector selection grid
│   ├── dashboard.html               # Screen 2: main dashboard with tabs
│   └── components/
│       ├── company_card.html
```

```
│       ├── signal_section.html
│       ├── news_item.html
│       ├── evidence_table.html
│       └── chat_panel.html
│
├── static/
│   ├── app.css                    # all styles
│   └── app.js                     # tab switching, chat AJAX, filter logic
│
├── data/
│   ├── cache/                     # diskcache for web fetches
│   ├── reports/                   # saved JSON reports
│   └── pulselens.db               # SQLite database
│
└── tests/
    ├── test_m3_extraction.py      # test with mock documents
    ├── test_m4_triangulation.py   # test corroboration + contradiction logic
    ├── test_m5_scoring.py         # test formula with mock claims
    └── test_chat.py               # test RAG + citation validation
```

Environment variables

bash

```
# Required
ANTHROPIC_API_KEY=
BRIGHTDATA_API_KEY=
BRIGHTDATA_SERP_ZONE=
BRIGHTDATA_SCRAPER_ZONE=
ALPHA_VANTAGE_API_KEY=       # free at alphavantage.co

# Optional
LOG_LEVEL=INFO
CACHE_TTL_HOURS=4
MAX_FACTS_PER_DOCUMENT=10
FINBERT_DEVICE=cpu           # or "cuda" if GPU available
```

---

12. Build order

Principles

1. **Schema first, code second** — finalize `models.py` before writing any module
2. **Test each module with mock data** — every module is independently testable
3. **Module 3 is the critical bottleneck** — validate extraction quality thoroughly before continuing
4. **Frontend can be built in parallel** — mock data → build UI → wire in real pipeline later

Detailed build sequence

```
FOUNDATION  (complete before any pipeline code)
  ☐ app/schemas/models.py          all Pydantic v2 models
  ☐ app/config/companies.py        company universe with full metadata
  ☐ app/config/signal_types.py     SignalType enum + SIGNAL_WEIGHTS
  ☐ app/config/source_tiers.py     domain→tier mapping + assign_tier()
  ☐ app/utils/helpers.py           uuid, date utils, text cleaning
  ☐ requirements.txt + .env.example
  ☐ SQLite schema setup (create_tables.py)

DATA PIPELINE  (in order — each depends on the previous)
  ☐ M1: m1_query_intelligence.py   test: print 20 queries to console
  ☐ M2: m2_web_collection.py       test: fetch 3 real queries, inspect raw HTML
  ☐ M3: m3_fact_extraction.py      CRITICAL: manually review extraction quality
  ☐ M3: finbert_client.py          test: sentiment on 10 sample claims
  ☐ M4: m4_triangulation.py        test: corroboration + contradiction with mock
facts
  ☐ M5: m5_scoring.py              test: verify formula output with mock claims
  ☐ M6: m6_intelligence_output.py  test: check narrative quality on real data

API LAYER
  ☐ POST /api/run                  trigger full pipeline run
  ☐ GET  /api/report/{id}          return MarketPulseReport as JSON
  ☐ POST /api/chat                 RAG chat over report evidence
  ☐ GET  /api/stock/{ticker}       Alpha Vantage wrapper

FRONTEND
  ☐ templates/sector_select.html   sector grid with Coming Soon overlays
  ☐ templates/dashboard.html       main layout, topbar, tab structure
  ☐ Tab: Overview                  pulse score, signals, narrative, watch list
  ☐ Tab: Companies                 company cards with narratives
  ☐ Tab: Signals                   collapsible signal sections with evidence
  ☐ Tab: News                      news feed with filter pills
  ☐ Tab: Evidence                  filterable table with export
  ☐ Chat panel                     AJAX chat with citation link-through
  ☐ Stock price context section

POLISH
  ☐ Loading states (pipeline takes 2–5 minutes)
  ☐ Error handling in UI
  ☐ Basic mobile responsiveness
  ☐ Export market brief as Markdown
```

---

13. Demo flow

Step 1: Sector selection

User lands on sector selection screen → sees 6 sectors → clicks "US AI Hardware" (the only live sector) → enters dashboard.

Step 2: Overview tab (first impression)

User immediately sees:

- Pulse score 78.3/100 — Heating up — +6.1 vs last week
- Top 5 signals with citations
- Market narrative (Layer 3): analyst-grade explanation of why the market is moving
- Watch list (Layer 4): 3 specific things to monitor next week

Step 3: Company deep dive

User clicks Companies tab → sees 8 company cards with individual narratives → Supermicro card is red with "Elevated Risk" badge → user is interested.

Step 4: Chat interaction

User types into the chat panel: *"Why is Supermicro elevated risk and what should I watch for?"*

Chat responds with citations [fact_e1] [fact_e2], explains conflicting signals, proposes a specific watch trigger.

Step 5: Evidence validation

User clicks on [fact_e1] in the chat response → jumps to Evidence tab, row highlighted → sees exact Reuters quote, confidence 0.71, published May 21 → trusts the intelligence.

---

14. What NOT to build for MVP

```
× GraphRAG / knowledge graph      too complex, add post-hackathon
× Historical trend comparison      requires multiple pipeline runs
× Stock price predictions          out of scope, potential legal issues
× Buy / sell / hold signals        not the product positioning
× Real-time streaming              batch run is sufficient for demo
× Multiple markets simultaneously  one market, done well
× Custom company list from user    hardcode 8 companies
× Fine-tuning any model            use off-the-shelf
× PDF export                       Markdown export is sufficient
× Authentication / multi-user      single-user demo
× Candlestick charts or technicals not a trading tool
× Slack / email alerts             nice-to-have, not core
× Multi-language support
```

15. References

Paper	Link	Applied to
Multi-HyDE Financial RAG	arXiv:2509.16369	M1 — query decomposition
Agentic RAG Survey	arXiv:2501.09136	Overall architecture
RAG for Fintech: Agentic Design	arXiv:2510.25518	M1 + M2
RASG: Business Document Extraction	arXiv:2405.20245	M3 — extraction
ClaimCheck: Fact-Checking via Web	ACL KnowledgeNLP 2025	M4 — triangulation
Complex Claim Verification in the Wild	arXiv:2305.11859	M4 — pipeline design
FinBERT	HuggingFace: ProsusAI/finbert	M3 — sentiment scoring
BloombergGPT	arXiv:2303.17564	Background reference
ReAct: Reasoning + Acting	arXiv:2210.03629	Agent orchestration pattern

Version 2.0 — Updated after product design sessions. This is a living document.